

02 — AI GAP ANALYSIS (Phase 4)

- **Sinh viên:** Ninh Văn Khải — MSSV **23127060**
- **Đối tượng review:** 3 spec và 5 Page Object do AI sinh ra ở Phase 3 (80 test), sau đó em bổ sung thêm

GAP-08 và GAP-09 phát hiện được ở Phase 8 (83 test)

Ở tài liệu này, em ngồi đọc lại toàn bộ code mà AI đã sinh ra cho em ở Phase 3, với tâm thế đi tìm lỗi chứ

không phải đi xác nhận là nó đúng. Em xin trình bày lại những điểm yếu em tìm được, nguyên nhân em cho là

AI đã sai ở đâu, cách em sửa, và bằng chứng cụ thể cho từng điểm.

1. Kết quả quét anti-pattern tự động

Đầu tiên em chạy một loạt lệnh quét để loại trừ các anti-pattern mà đề bài và `SKILL.md` cấm. Em làm bước

này bằng lệnh thay vì đọc mắt, vì đọc mắt rất dễ bỏ sót:

```
grep -rn "waitForTimeout" tests/           => chỉ khớp trong COMMENT, 0 lời gọi thật
grep -rn "locator('.|locator(\"\".\" tests/ => 0 (không dùng class Tailwind)
grep -rn "nth-child|xpath=" tests/         => 0
grep -rn "^const cases = \[" tests/*.spec.js => 0 (không hardcode data inline)
npx playwright test --list --project=chromium => Total: 80 tests in 3 files
```

Kết quả cho thấy bộ test không vi phạm anti-pattern nào ở mức cú pháp. Nhưng em xin lưu ý là những lệnh

trên chỉ bắt được lỗi *hình thức*. Các điểm yếu thật sự về mặt logic thì em phải tự đọc mới ra, và em trình

bày chúng ở mục tiếp theo.

2. Bảng Gap Analysis

Em tìm được tổng cộng 9 điểm yếu, đánh số từ GAP-00 đến GAP-09. Với mỗi điểm, em cố gắng truy ngược lại

vì sao AI lại sai ở đó, vì em nghĩ hiểu được nguyên nhân thì lần sau mới tránh được:

#	Vấn đề	Loại	AI sai vì sao	Cách sửa
GAP-00	<code>getByRole('textbox')</code> ở bước 2 của <code>/forgot-password</code> khớp 2 phần tử (ô OTP + ô mật khẩu) → <code>strict mode violation</code> , 9/30 test FR-03 fail	fragile selector	Model limitation. AI khẳng định chắc nịch trong Page Object rằng "Playwright không expose role cho <code>input[type=password]</code> ". Đây là kiến thức API sai — Playwright có gán role <code>textbox</code> cho input password. AI viết comment giải thích cho một giả định chưa hề kiểm chứng.	Đổi sang <code>.first()</code> (ô OTP đứng trước trong DOM), giữ <code>input[type="password"]</code> cho ô mật khẩu và ghi lại nguyên nhân trong comment
GAP-01	<code>expect(c.expect.minRows).toBe(SEED_PRODUCT_COUNT)</code> — so hằng số với hằng số, không chạm tới SUT	weak assertion	Prompt quality. Yêu cầu "≥3 assertion pattern mỗi test" khiến AI nhồi thêm assertion cho đủ số lượng thay vì đủ ý nghĩa. Đây đúng là anti-pattern <code>expect(true).toBe(true)</code> trá hình.	Giữ dòng đó nhưng hạ xuống vai trò "kiểm tra data file khớp seed", thêm assertion thật: số dòng trên UI phải bằng <code>GET /api/products .length</code>
GAP-02	FR15-TC05 đếm <code>totalRows</code> từ rất sớm rồi mới thao tác sửa — giữa hai mốc đó có nhiều bước, worker song song có thể thêm/xoá sản phẩm	flaky wait	Đặc thù feature. Test chạy <code>fullyParallel</code> trên một CSDL SQLite dùng chung. AI viết test như thể mình đọc chiếm dữ liệu.	Dời phép đếm xuống ngay trước khi bấm Lưu, và chờ dòng mục tiêu xuất hiện trước đó
GAP-03	<code>expect(xssExecuted).toBe(!c.expect.scriptDidNotExecute)</code> — phủ định kép	weak assertion	Model limitation. AI cố ép mọi giá trị kỳ vọng phải đọc từ data file, kể cả khi việc đó làm assertion không đọc nổi. Người review rất dễ hiểu ngược.	Viết thẳng <code>.toBe(false)</code> cho hành vi, tách riêng 1 dòng đối chiếu data file
GAP-04	<code>openProductsTab()</code> chỉ chờ nút "Lưu sản phẩm" — nút đó thuộc form, có ngay lập tức, trong khi <code>fetchData()</code> vẫn đang chạy → test đếm dòng có thể đọc bảng rỗng	flaky wait	Đặc thù feature. SPA render form và bảng độc lập nhau; AI chọn phần tử "thấy được sớm nhất" làm mốc chờ mà không phân biệt phần tử nào phụ thuộc dữ liệu.	Chờ thêm nút <code>Sửa</code> đầu tiên (chỉ tồn tại khi đã có dữ liệu dòng)
GAP-05	FR08-TC01 chỉ kiểm tra "giỏ có tiền > 0", không hề kiểm tra đúng sản phẩm nào đã vào giỏ	missing edge case	Prompt quality. Case trong data file mô tả "happy path checkout" nên AI tối ưu cho việc đi hết luồng, bỏ qua tính đúng đắn của dữ liệu giữa chừng.	Ghi lại tên sản phẩm ở Home rồi assert dòng đó có mặt trong bảng giỏ hàng; thêm <code>rowByProductName()</code> vào <code>CartPage</code>
GAP-06	Biến thể token 9999 trong CSV có xác suất 1/9000 trùng token thật → test kỳ vọng 400 sẽ ngẫu nhiên nhận 200	flaky wait	Đặc thù feature. Token của SUT chỉ 4 chữ số nên không gian trùng rất nhỏ. AI ban đầu coi "token bịa ra" là chắc chắn sai.	Xin lại token cho tới khi khác giá trị biến thể, kèm comment nêu rõ đây là chống trùng chứ không phải retry che lỗi
GAP-07	Trình duyệt <code>firefox / webkit</code> chưa được tải → toàn bộ test UI fail sau 3 ms với <code>browserType.launch: Executable doesn't exist</code>	môi trường	Model limitation. AI mặc định môi trường đã sẵn sàng vì chromium chạy được. Kết quả "52 passed" trông như thành công một phần, thực chất là 28 test không chạy nổi.	<code>npm playwright install firefox webkit</code> trước khi vào Phase 5
GAP-08	<code>CartPage.itemRowCount()</code> gọi <code>isVisible()</code> ngay sau khi điều hướng — <code>isVisible()</code> không chờ, nên khi React chưa render xong thì hàm trả về <code>0 - 1 = -1</code>	flaky wait	Đặc thù feature + model limitation. AI dùng <code>isVisible()</code> như một phép kiểm tra tức thời mà quên rằng nó không có cơ chế chờ như <code>expect(...).toBeVisible()</code> . Trên chromium đủ nhanh nên không lộ; chỉ webkit mới làm fail.	Chờ trang chốt trạng thái bằng <code>expect(emptyMessage.or(cartHeading)).toBeVisible()</code> rồi mới đếm
GAP-09	FR15-TC05 đếm số dòng ngay sau <code>await dialogPromise</code> — nhưng đóng alert không đồng nghĩa React đã render lại bảng	flaky wait	Đặc thù feature. AI coi "alert đã đóng" là mốc an toàn để đọc DOM. Thực tế <code>setProducts()</code> chạy trước alert nhưng React render bất đồng bộ sau đó.	Chờ <code>expect(rowByName(newName).first()).toBeVisible()</code> rồi mới đếm

3. Bốn lỗi AI đáng chú ý nhất (dùng cho AI_Critique)

Từ 9 điểm yếu ở trên, em thử gom lại thành bốn kiểu sai mà em thấy lặp đi lặp lại. Em nghĩ đây mới là phần

đáng rút kinh nghiệm, vì nó nói lên cách AI hay sai chứ không chỉ là từng lỗi lẻ.

1. **AI rất tự tin về một API mà nó không nắm chắc (GAP-00).** Với em, điều nguy hiểm nhất ở đây không phải bản thân cái sai, mà là việc AI **viết hẳn một comment giải thích** cho cái sai đó. Người đọc nhìn vào sẽ tưởng rằng giả định đã được kiểm chứng rồi nên không kiểm lại nữa. Em chỉ phát hiện ra khi chạy thật.

1. **AI đếm số lượng assertion thay vì quan tâm tới chất lượng (GAP-01, GAP-03).** Khi rubric nói "≥3

assertion pattern", AI có xu hướng làm cho đủ con số bằng những assertion thực chất không kiểm thử gì cả.

1. **AI bỏ qua tính đồng thời và trạng thái môi trường (GAP-02, GAP-04, GAP-07).** Nó viết test như thể chỉ

có một mình nó đang chạy, trên một cái máy đã được cài đặt hoàn hảo từ trước.

1. **AI chỉ kiểm chứng trên trình duyệt nhanh nhất (GAP-08, GAP-09).** Hai lỗi về cơ chế chờ này **pass sạch

trên chromium, kể cả khi em chạy với `--repeat-each=2` **, và chỉ lộ ra khi em chạy webkit ở Phase 8.

Bài học em rút ra là "ổn định 2 lần liên tiếp trên 1 browser" **không** đồng nghĩa với việc test đã ổn

định. Cả hai lỗi này đều bắt nguồn từ cùng một thói quen: lấy phép đọc DOM **không có cơ chế chờ** như

`isVisible()` hay `count()` làm móc, thay vì dùng web-first assertion.

4. Kết quả sau khi sửa

Sau khi vá hết 9 điểm yếu ở trên, em chạy lại để xác nhận. Em xin dán nguyên văn output thật của các lệnh

em đã chạy:

```
npx playwright test --project=chromium
⇒ 80 passed (17.0s)      ← trước khi sửa: 80 passed nhưng có 5 điểm yếu ở bảng trên

# Sau khi bổ sung test ở Phase 8 và vá GAP-08 / GAP-09:
npx playwright test --project=webkit
⇒ 83 passed (35.8s)

npx playwright test --project=webkit --grep "FR08-TC17|FR15-TC05" --repeat-each=3   (chạy 3 lượt)
⇒ 6 passed · 6 passed · 6 passed   ← 18/18, hai test từng flaky nay ổn định

node scripts/run-multibrowser.mjs
⇒ 9/9 run PASS toàn bộ (249 test)
```

Riêng với hai test từng bị flaky là `FR08-TC17` và `FR15-TC05`, em cố ý chạy lặp 3 lượt trên webkit — đúng

trình duyệt đã làm chúng fail — để chắc chắn rằng em đã sửa đúng nguyên nhân chứ không phải chỉ may mắn.

Chi tiết số liệu của cả 9 lần chạy multi-browser, em trình bày ở `report/03-RUN-SUMMARY.md` (Phase 5).

5. Xác nhận review script của người học

- [x] Đã **đọc lại 9 GAP** ở §2 và xác nhận cách sửa hợp lý — kể cả GAP-08 / GAP-09 là hai lỗi chỉ lộ ra

trên WebKit, đúng như phần "AI chỉ kiểm chứng trên trình duyệt nhanh nhất" đã phân tích.

- [x] **Quyết định về GAP-01 / GAP-03:** *giữ* các assertion đối chiếu data file, nhưng chỉ ở dạng đối chiếu

giá trị thật lấy từ UI/API với giá trị kỳ vọng trong data file — đã bỏ hết kiểu `expect(hàng).toBe(hàng)`

vốn không kiểm thử gì.

- [x] **Ký xác nhận "đã review script":** Ninh Văn Khải — 23127060 · **Ngày:** 2026-08-27